MULTIPLAYER GAME CODE ARCHITECTURE DESIGN

Game: PUBG

Submitted By:
Ahmed Suleman

SAP ID: 57141

BSCS 5B

Game Overview:

PUBG is a multiplayer online battle royale game where up to 100 players compete against each other on a large map. Players land on the battlefield, collect weapons, equipment, and supplies, and fight to become the last surviving player or team.

The game combines survival, exploration, looting, and combat mechanics. As the match progresses, a shrinking safe zone forces players into smaller areas, increasing player encounters and making the gameplay more intense.

Core Gameplay Mechanics:

- Player Movement
- Weapon Collection
- Inventory Management
- Vehicle Usage
- Combat System
- Safe Zone System
- Team Communication
- Matchmaking

High-Level Architecture:

Player System:
The Player System manages player movement, health, animations, statistics, and player actions throughout the match.

Matchmaking System:
The Matchmaking System creates game lobbies, matches players together, and starts game sessions.

Inventory System:
The Inventory System manages weapons, ammunition, healing items, armor, and player equipment.

Loot System:
The Loot System handles spawning and distribution of weapons and resources across the map.

Combat System;
The Combat System manages shooting mechanics, damage calculations, hit detection, and kill registration.

Vehicle System:
The Vehicle System controls vehicle spawning, movement, fuel usage, and vehicle damage.

Safe Zone System:
The Safe Zone System continuously reduces the playable area and applies damage to players outside the zone.

Network System:
The Network System synchronizes player actions and game data between the server and connected clients.

UI System:
The UI System displays health bars, maps, inventory screens, player statistics, and match results.

Code Structure

The PUBG architecture follows a modular design where each system is responsible for a specific functionality. This improves maintainability and scalability.

Main Classes;
GameManager
• Start Match
• End Match
• Manage Game States
• Determine Winners

PlayerManager
• Spawn Players
• Track Health
• Manage Player Statistics

InventoryManager
• Store Items
• Equip Weapons
• Manage Consumables
LootManager
• Spawn Loot
• Manage Pickups

CombatManager
• Process Shooting
• Calculate Damage
• Register Kills

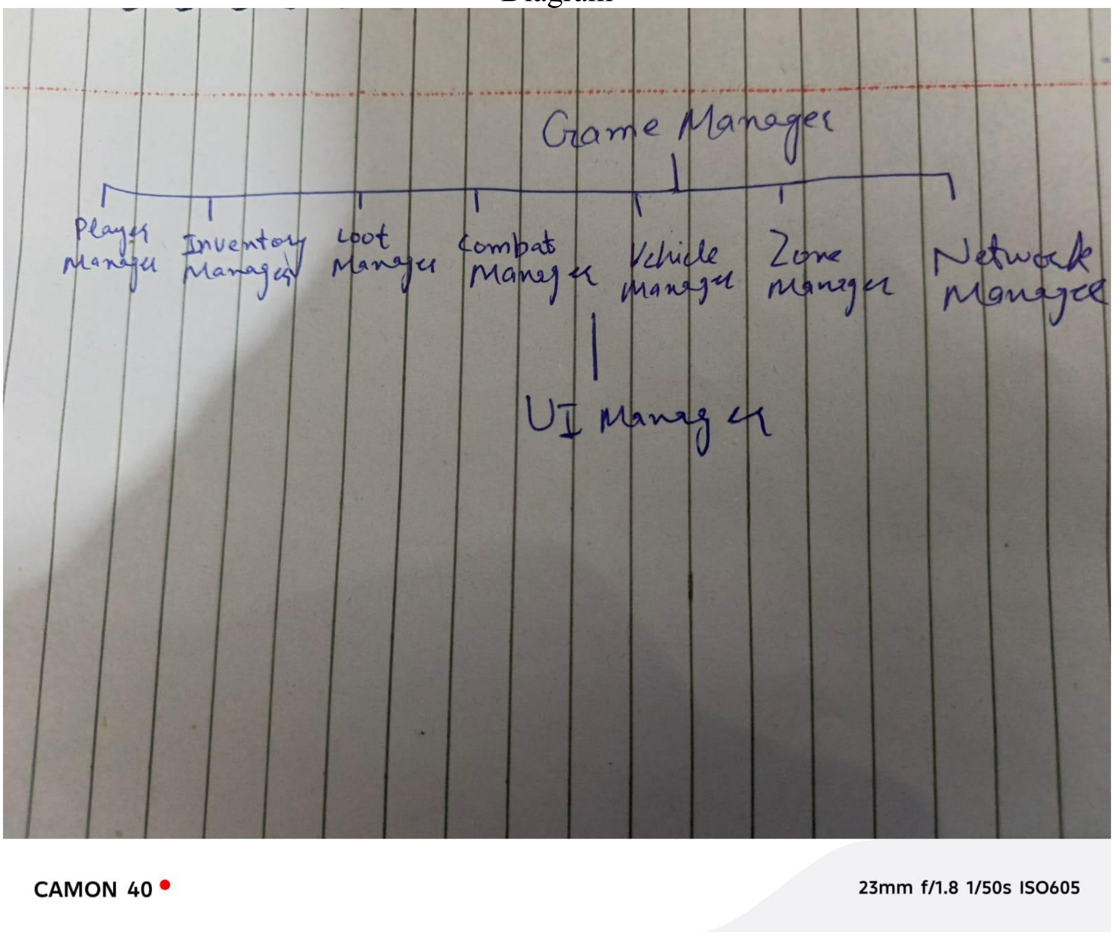
VehicleManager
• Spawn Vehicles
• Handle Driving
• Manage Vehicle Health

SafeZoneManager
• Shrink Zone
• Apply Zone Damage

NetworkManager
• Client-Server Communication
• Data Synchronization

UIManager
• Display HUD
• Show Inventory
• Update Minimap

Diagram



Design Decisions

This architecture uses a modular approach where each system has a dedicated responsibility. The separation of systems reduces code complexity and improves maintainability.

A manager-based architecture allows developers to modify or update individual systems without affecting the entire project. The NetworkManager is isolated from gameplay systems, making multiplayer synchronization more reliable.

This architecture is highly scalable and suitable for large multiplayer games such as PUBG because it supports future expansion, efficient debugging, and organized development.